LESSON 6

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Make Some Noise

Beeps, sirens, songs, and a piano you build yourself.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between an **active** and a **passive** buzzer
- How sound is made by **vibrations** (frequency)
- How to play notes with `tone()` and `noTone()`
- How to store a song in two **arrays**

SESSION PLAN

0:00	Warm-up: what is sound?
0:10	Sketch 1: **L06_Beep** (active buzzer)
0:25	Swap to the passive buzzer, learn `tone()`
0:35	Sketch 2: **L06_Melody**
0:55	Short break
1:00	Build the 5-button keyboard
1:15	Sketch 3: **L06_Piano**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 1 active buzzer (sealed bottom, sticker on top)
- ☐ 1 passive buzzer (open bottom, you can see the green board)
- ☐ 5 push buttons
- ☐ About 8 jumper wires

1. What is sound?

Sound is air **vibrating**. A speaker or buzzer pushes the air back and forth very fast, and your ear feels those pushes. The number of pushes per second is the **frequency**, measured in **hertz** (Hz).

Frequency	Sounds like
100 Hz	a low hum
440 Hz	the note A, used to tune instruments
2000 Hz	a high beep
15000 Hz	a whine that only young ears can hear!

Your kit has two kinds of buzzer:

	Active buzzer	Passive buzzer
Looks like	Sealed bottom, often a sticker on top	Open bottom, you can see a green board
Inside	Its own little beep-maker	Just the part that vibrates
How to use it	Switch it on: it beeps (one sound only)	You make the vibrations: any note you like

2. Sketch 1: Beep!

Find the **active** buzzer. Its **+** leg is the longer one (there's often a **+** on top too).

✓	From	To
☐	Active buzzer, in the breadboard	each leg in its own row
☐	ESP32 pin **33**	buzzer **+** leg (longer)
☐	buzzer other leg	**–** rail (GND)

File → **New Sketch**, save it as **L06_Beep**, and upload.

Type this in → save it as **L06_Beep** — 22 lines

```
// Lesson 6: Beep with the active buzzer
// Active buzzer: + leg to pin 33, other leg to GND.

const int BUZZER = 33;

void beep(int length) {
  digitalWrite(BUZZER, HIGH);
  delay(length);
  digitalWrite(BUZZER, LOW);
  delay(150);
}

void setup() {
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  beep(100);
  beep(100);
  beep(400);
  delay(2000);
}
```

This is just like blinking an LED, but with sound. The `beep()` function takes one number, the **length** of the beep in milliseconds.

> **Try it**
>
> Change the `beep` lines in `loop()` to beep your name in Morse code, or make a doorbell sound.

3. The passive buzzer and tone()

Now swap the active buzzer for the **passive** one, in exactly the same place (**+** leg to pin 33).

> **Experiment**
>
> Upload **L06_Beep** again with the passive buzzer. What do you hear? Why?

The passive buzzer only **clicks** when the pin switches on or off. To make a note, the pin has to switch on and off hundreds of times a second. The `tone()` command does that for you:

Code	What it means
`tone(BUZZER, 440);`	Vibrate pin 33 at 440 Hz (the note A), until we say stop
`noTone(BUZZER);`	Stop the sound

Here are the notes you'll need today:

Note	C	D	E	F	G	A	B	high C
Frequency (Hz)	262	294	330	349	392	440	494	523

4. Sketch 2: Play a melody

A song is a list of **notes** and how long each note lasts (its **beats**). We'll keep each list in its own array. **File → New Sketch**, save it as **L06_Melody**.

Type this in → save it as **L06_Melody** — 23 lines

```
// Lesson 6: Play a tune on the passive buzzer
// Passive buzzer: + leg to pin 33, other leg to GND.

const int BUZZER = 33;
const int TEMPO = 300;

int notes[] = {262, 262, 392, 392, 440, 440, 392,
               349, 349, 330, 330, 294, 294, 262};
int beats[] = {1, 1, 1, 1, 1, 1, 2,
               1, 1, 1, 1, 1, 1, 2};

void setup() {
}

void loop() {
  for (int i = 0; i < 14; i++) {
    tone(BUZZER, notes[i]);
    delay(beats[i] * TEMPO);
    noTone(BUZZER);
    delay(50);
  }
  delay(2000);
}
```

Code	What it means
`int notes[] = { … };`	The frequency of each note, in order
`int beats[] = { … };`	How many beats each note lasts (1 = short, 2 = long)
`const int TEMPO = 300;`	One beat lasts 300 ms. Smaller = faster song
`notes[i]` and `beats[i]`	The same `i` gives a note **and** its length
`delay(50);`	A tiny gap so repeated notes don't blur together

Big idea: data and code

The `for` loop is the **code**: it knows *how* to play. The arrays are the **data**: *what* to play. To play a different song you only change the data!

Arrays must match

Both arrays need the same number of items, and the `14` in the `for` loop must match. If you add notes, count them carefully.

Name that tune

What song is it? Can you add the next line of the song?

---

5. Build a 5-button keyboard

Put 5 buttons in a row across the middle gap, like piano keys. Each button connects one ESP32 pin to GND, just like in Lesson 3. Keep the passive buzzer on pin 33.

✓	Button	ESP32 pin	Diagonal leg to	Note
☐	1	**13**	– rail	C
☐	2	**14**	– rail	D
☐	3	**16**	– rail	E
☐	4	**17**	– rail	F
☐	5	**18**	– rail	G

Running out of room?

Take the RGB LED and the potentiometer off the breadboard for now. Use short wires and leave one empty row between buttons.

6. Sketch 3: Button piano

File → New Sketch, save it as **L06_Piano**.

Type this in → save it as **L06_Piano** 34 lines

```
// Lesson 6: Button piano
// Passive buzzer on 33. Five buttons on 13, 14, 16, 17, 18
// (other legs to GND) play C, D, E, F, G.

const int BUZZER = 33;

int keys[] = {13, 14, 16, 17, 18};
int freqs[] = {262, 294, 330, 349, 392};
int lastNote = 0;

void setup() {
  for (int i = 0; i < 5; i++) {
    pinMode(keys[i], INPUT_PULLUP);
  }
}

void loop() {
  int note = 0;
  for (int i = 0; i < 5; i++) {
    if (digitalRead(keys[i]) == LOW) {
      note = freqs[i];
    }
  }

  if (note != lastNote) {
    if (note > 0) {
      tone(BUZZER, note);
    } else {
      noTone(BUZZER);
    }
    lastNote = note;
  }
  delay(10);
}
```

Code	What it means
`int keys[] = { … };`	Which pin each key is on
`int freqs[] = { … };`	The note for each key
`int note = 0;`	0 means "no key is pressed"
`if (note != lastNote)`	Only change the sound when a different key is pressed (or let go)

Play it

Try *Mary Had a Little Lamb*: **E D C D E E E**, **D D D**, **E G G**.

7. Challenges

Challenge 1: Your own song

Change the arrays in **L06_Melody** to play a different song. Search for the notes of a song you like, or compose your own!

Challenge 2: Police siren

Make the pitch slide up and down like a siren. *Hint: use a* `for` *loop that counts the frequency from 500 to 1200, calling* `tone()` *with each value.*

Challenge 3: Light-up piano

Put the RGB LED back on pins 25, 26 and 27 (Lesson 5). Make each key light up a different colour while it plays.

Bonus: Knob pitch

Put the potentiometer back on pin 34 and use `map()` so turning the knob changes the pitch, like a theremin.

8. Quiz

Check what you learned

1. Which buzzer can play different notes: active or passive?

   _____

2. What does `tone(BUZZER, 262);` do? How do you stop it?

   _____

   _____

3. If an array has 14 items, what number is the last item?

   _____

Tidy up: unplug the USB cable. Put both buzzers and the buttons back in the kit, and leave the breadboard clear for Lesson 7.